

Inner class

- ❖ When you define a class within another class then it is called as inner class or nested class.

Example: -

```
class Outer {  
    ....  
    class Inner {  
        ....  
    }  
}
```

Type of Inner class

- I. Instance Inner class
- II. Static Inner class
- III. Local Inner class
- IV. Anonymous class

I. Instance Inner class

- ❖ When you define a class within another class without static modifier is called as instance inner class.
- ❖ Syntax to create Instance inner class object from outside the outer class.
Outer. Inner outob=new Outer ().new Inner ();
Or
Outer outob =new Outer ();
Outer. Inner inob = outob. new Inner ();
- ❖ Instance Inner class member cannot be accessed from outer class directly but it can be accessed from outer class with object of instance inner class.
- ❖ All the static and instance member of Outer class can be accessed inside the instance inner class directly.
- ❖ You **can** use instance inner class name directly in outer class but **cannot** use instance inner class name directly from outside the outer class.
- ❖ Instance Inner class cannot contain declaration i.e. you **cannot** define static variables or static methods inside the instance inner class.
- ❖ You can declare final static variable inside the Instance Inner class.
- ❖ Outer class can be either default or public.
- ❖ Inner class can be private or public or protected or static.
- ❖ Inner class can be abstract and you can use private and abstract combination with inner class.
- ❖ Outer class can contains interfaces also.
- ❖ Inner class can extend another inner class or it can implement another inner interface.
- ❖ Inner class can be nested i.e. you can write inner class inside another inner class.
- ❖ An interface can contain another inner interface or inner class.
- ❖ Top level class can contain inner interface but class cannot contain inner interface.

- ❖ You can use **this** keyword with Outer class name to access the members of Outer class when the same member available in inner class and locally.
- ❖ Usually private member cannot be accessed from outside the class but private member of inner class can be accessed from outer class with object of inner class.

Program:**OOPS435.java**

```

class Outer{
    int a=10;
    static int b=20;
    void m1(){
        System.out.println("outer m1()");
    }
    static void m2() {
        System.out.println("outer m2()");
    }
    class Inner {
        int x=1;
        void show(){
            System.out.println("Inner show()");
        }
    }
}

```

```

System.out.println(a);
System.out.println(b);
System.out.println(x);
m1();
m2();
}
}
class OOPS435{
    public static void main(String arg[]){
        Outer ob=new Outer();
        ob.m1();
        ob.m2();
        Outer.Inner iob=new Outer().new Inner();
        iob.show();
    }
}

```

OOPS436.java

```

class Outer {
    void show(){
        System.out.println("outer show()");
        System.out.println(x);
        m1();
    }
    class Inner {
        int x=11;
        void m1() {
            System.out.println("Inner m1()");
        }
    }
}
class OOPS436{
    public static void main(String args[]){
        Outer ob=new Outer();
        ob.show();
    }
}

```

OOPS437.java

```

class Outer{
    void show() {
        System.out.println("outer show()");
        Inner inob=new Inner();
        System.out.println(inob.x);
        inob.m1();
    }
    class Inner {
        int x=11;
        void m1(){
            System.out.println("Inner m1()");
        }
    }
}
class OOPS437{
    public static void main(String args[]){
        Outer ob=new Outer();
        ob.show();
    }
}

```

OOPS438.java

```

class Outer{
    class Inner {
        static int a=11;
        static void m1(){
        }
    }
}
class OOPS438{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS439.java

```

class Outer{
    class Inner{
        final static int a=11;
    }
}
class OOPS439{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS440.java

```

class Outer{
    class Inner {
        int a=10;
        void m1(){
        }
    }
}
class OOPS440{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS441.java

```

class Outer{
    private class Inner1{}
    protected class Inner2{}
    public class Inner3{}
    static class Inner4{}
    abstract class Inner5{}
    private abstract class Inner6{}
}
class OOPS441{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS442.java

```

class Outer{
    private class Inner1{}
    protected class Inner2{}
    public interface Inner3{}
    static interface Inner4{}
    abstract interface Inner5{}
    private abstract interface Inner6{}
}
class OOPS442{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS443.java

```

class Outer{
    interface Inner1{}
    interface Inner2 extends Inner1{}
    class Inner1 implements Inner2{}
    class Inner2 extends Inner1{}
    private abstract class Inner3{}
    class Hello extends Inner3{}
}
class OOPS443{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS444.java

```

class Outer{
    class Inner1{
        class Inner2{
            class Inner3{
                class Inner4{
                    class Inner5{}
                }
            }
        }
    }
}
class OOPS444{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS445.java

```

class Outer{
    int a=10;
    class Inner{
        int a=22;
        void show() {
            System.out.println("Inner -> show()");
            int a=33;
            System.out.println(a);
            System.out.println(this.a);
            System.out.println(Inner.this.a);
            //System.out.println(super.a);
            System.out.println(Outer.this.a);
        }
    }
}
class OOPS445{
    public static void main(String arg[]){
        new Outer().new Inner().show();
    }
}

```

OOPS446.java

```

class Outer{
    private int a=10;
    class Inner{
        private int a=20;
        void showInner () {
            System.out.println("Inner->
showInner()");
            System.out.println(a);
            System.out.println(this.a);
        }
    }
}

```

```

void showOuter () {
    System.out.println("Inner ->
showOuter()");
    System.out.println(a);
    System.out.println(new Inner().a);
}
}
class OOPS446{
    public static void main(String arg[]){
        new Outer().new Inner().showInner();
        new Outer().showOuter();
    }
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS435	outer m1() outer m2() Inner show() 10 20 11 outer m1() outer m2()	All the static and instance member of Outer class can be accessed inside the instance inner class directly. Here, variable, void m1 () and static void m2 () are outer class member so we can be accessed inside the instance inner class directly.
OOPS436	Compilation error	[error: cannot find symbol x] Instance Inner class member cannot be accessed from outer class directly.
OOPS437	outer show() 11 Inner m1()	Instance Inner class can be accessed from outer class with object of instance inner class. Here, we are creating object for inner class.
OOPS438	Compilation error	[Error: Illegal static declaration in inner class Outer. Inner static int a=11; modifier 'static' is only allowed in constant variable declarations] Instance Inner class cannot contain declaration i.e. you cannot define static variables or static methods inside the instance inner class.
OOPS439	Hello guys!	You can declare final static variable inside the Instance Inner class.
OOPS440	Hello guys!	You can declare Instance variable inside the Instance Inner class.
OOPS441	Hello guys!	Outer class can contain public, protected, private, static, abstract and private abstract.
OOPS442	Hello guys!	Outer class can contains interfaces also.
OOPS443	Compilation error	[error: interface Inner1 is already defined in class Outer And interface Inner2 is already defined in class Outer]

Prog No.	Output	Error and Details
OOPS444	Hello guys!	Inner class can be nested
OOPS445	Inner -> show() 33 22 22 10	You can use this keyword with Outer class name to access the members of Outer class when the same member available in inner class and locally. But you cannot use super for access member of Outer class.
OOPS446	Inner -> showInner() 20 20 Inner -> showOuter() 10 20	Usually private member cannot be accessed from outside the class but private member of inner class can be accessed from outer class with object of inner class.

II. Static Inner class

- When you define a class within another class with static modifier is called as static inner class.
- Syntax to create static inner class object from outside the outer class.
Outer . Inner outob=new Outer.Inner ();
- Static member of Outer class can access inside the static inner class.
- Instance member of Outer class can be accessed with the Outer class object.
- Static inner class can contain declarations i.e. you **can** define static variables or static methods inside the static inner class.
- Static inner class member **cannot** be accessed from outer class directly.
- Static inner class member **can** be accessed from outer class with class name or object of static inner class.
- You can use static inner class name directly in outer class.
- You cannot use static inner class name directly from outside the Outer class.
- You can use following Syntax to access instance inner class outside the outer class.

Outer . Inner inob=new Outer.Inner ();

Program:

OOPS447.java <pre> class Outer{ int a=10; static int b=20; void m1(){ System.out.println("outer m1()"); } static void m2() { System.out.println("outer m2()"); } class Inner{ int x=11; static int y=22; void show(){ System.out.println("Inner show()"); System.out.println(a); System.out.println(b); System.out.println(x); } </pre>	<pre> System.out.println(y); m1(); m2(); } static void m3(){ System.out.println("Inner => m3() "); } } class OOPS447{ public static void main(String arg[]){ Outer ob=new Outer(); ob.m1(); ob.m2(); Outer.Inner iob=new Outer.Inner(); iob.show(); iob.m3(); } } </pre>
---	---

OOPS448.java

```

class Outer{
int a=10;
static int b=20;
void m1(){
    System.out.println("Outer m1()");
}
static void m2(){
    System.out.println("outer m2()");
}
static class Inner{
    int x=11;
    static int y=22;
    void show(){
        System.out.println("Inner show()");
        System.out.println(b);
        System.out.println(x);
    }
}

```

```

System.out.println(y);
m2();
}
static void m3(){
    System.out.println("Inner => 30");
}
}
}
class OOPS448{
    public static void main(String arg[]){
        Outer outob=new Outer();
        outob.m1();
        outob.m2();
        Outer.Inner inob=new Outer.Inner();
        inob.show();
        inob.m3();
    }
}

```

OOPS449.java

```

class Outer{
void show(){
    System.out.println("outer=>showOuter()");
    System.out.println(x);
    System.out.println(y);
    m1();
}
static class Inner{
    int x=11;
    static int y=22;
    static void m1(){
        System.out.println("Inner m1()");
    }
}
}
class OOPS449{
    public static void main(String arg[]){
        Outer ob=new Outer();
        ob.show();
    }
}

```

OOPS450.java

```

lass Outer{
void show(){
    System.out.println("outer=>showOuter()");
    System.out.println(new Inner().x);
    System.out.println(Inner.y);
    Inner.m1();
}
static class Inner{
    int x=11;
    static int y=22;
    static void m1(){
        System.out.println("Inner m1()");
    }
}
}
class OOPS450{
    public static void main(String arg[]){
        Outer ob=new Outer();
        ob.show();
    }
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS447	Compilation error	[Error: Illegal static declaration in inner class Outer.Inner modifier 'static' is only allowed in constant variable declarations] Refer page 165 point 1 to 5
OOPS448	Outer m1() outer m2()	Refer page 165 point 1 to 5

	Inner show() 20 11 22 outer m2() Inner => 3()	
OOPS449	Compilation Error	[Error: cannot find symbol x, y;] Refer page 165 point 6 to 10
OOPS450	outer=>show() 11 22 Inner m1()	Refer page 165 point 6 to 10

I. Local Inner class

1. When you define class within methods, blocks or constructors is called as Local inner class.
2. Local Inner class must be accessed from the same methods, blocks or constructors where it is define.
3. When you are writing Local Inner class inside the Instance method then you can access both instance and static members of Outer class.
4. When you are writing Local Inner class inside the static method then you can access only static members of Outer class.
5. When you want to access local variable of block or method or constructor inside the local inner class then variable needs to be declared as final

Programs:

OOPS451.java

```

class Hello{
{
    System.out.println("I. B");
    class Inner{}
}
static{
    System.out.println("S. B");
    class Inner{}
}
Hello(){
    System.out.println("D. Cons");
    class Inner{}
}
void m1(){
    System.out.println("m1()");
    class Inner{}
}
}
class OOPS451{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}
  
```

OOPS452.java

```

class Outer{
    int a=10;
    static int b=20;
    void show(){
        System.out.println("Outer->show()Begin");
        class Inner{
            void m1(){
                System.out.println("Inner => m1()");
                System.out.println(a);
                System.out.println(b);
            }
        }
        new Inner().m1();
        System.out.println("Outer=>show() End");
    }
}
class OOPS452{
    public static void main(String arg[]){
        Outer out=new Outer();
        out.show();
    }
}
  
```


OOPS453.java

```

class Outer{
int a=10;
static int b=20;
static void show(){
System.out.println("Outer =>show ()Begin");
class Inner{
void m1(){
System.out.println("Inner => m1()");
System.out.println(a);
System.out.println(b);
}
}
new Inner().m1();
System.out.println("Outer=>show() End");
}
}
class OOPS453{
public static void main(String arg[]){
Outer. show();
}
}

```

OOPS454.java

```

class Outer{
int a=10;
static int b=20;
static void show(){
System.out.println("Outer-show()Begin");
class Inner{
void m1(){
System.out.println("Inner => m1()");
System.out.println(b);
}
}
new Inner().m1();
System.out.println("Outer-show() End");
}
}
class OOPS454{
public static void main(String arg[]){
Outer. show();
}
}

```

OOPS455.java

```

class Outer{
void show(){
System.out.println("Outer->show()");
class Inner{
int a=10;
static int b=20;
final static int c=30;
void m1(){
System.out.println("Inner => m1()");
}
static void m2(){
System.out.println("Inner => m2()");
}}}}
class OOPS455{
public static void main(String arg[]){
System.out.println ("Hello Guys!");
}
}

```

OOPS456.java

```

class Outer{
void show(){
System.out.println("Outer->show()");
class Inner{
int a=10;
final static int c=30;
void m1(){
System.out.println("Inner => m1()");
}
}
}
}
class OOPS456{
public static void main(String arg[]){
System.out.println ("Hello Guys!");
}
}

```

OOPS457.java

```

class Outer{
void m1(){
class Inner1{
}
void m2(){
class Inner2{
}
void m3(){

```

```

Inner1 ob1=new Inner1();
Inner2 ob2=new Inner2();
}
}
class OOPS457{
public static void main(String arg[]){
System.out.println ("Hello Guys!");
}
}

```


<u>OOPS458.java</u> <pre> class Outer{ void m1(){ //Inner1 ob1=new Inner1(); class Inner1{} Inner1 ob2=new Inner1(); } </pre>	<pre> void m2(){ class Inner2{} Inner2 ob2=new Inner2(); } } class OOPS458{ public static void main(String arg[]){ System.out.println ("Hello Guys!"); }} </pre>
<u>OOPS459.java</u> <pre> class Outer{ void m1(){ int a=10; class Inner{} void show(){ System.out.println(a); }} new Inner().show(); } class OOPS459{ public static void main(String arg[]){ new Outer().m1(); }} </pre>	<u>OOPS460.java</u> <pre> class Outer{ void m1(){ final int a=10; class Inner{ void show(){ System.out.println(a); }} new Inner().show(); } class OOPS460{ public static void main(String arg[]){ new Outer().m1(); }} </pre>

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS451	Hello guys!	Refer page 169 point 1 and 2
OOPS452	Outer->show()Begin Inner => m1() 10 20 Outer=>show() End	Refer page 169 point 3
OOPS453	Compilation error	[Error: Cannot make a static reference to the non-static field a] Refer page 169 point 4
OOPS454	Outer->show()Begin Inner => m1() 20 Outer=>show() End	Refer page 169 point 4
OOPS455	Compilation error	[Error: Illegal static declaration in inner class Inner] Refer page 169 point 5
OOPS456	Hello Guys!	Refer page 169 point 5
OOPS457	Compilation error	[Error: cannot find symbol Inner1] we can create object in same method scope.
OOPS458	Hello Guys!	we can create object in same method scope.
OOPS459	Compilation error	[Error: Cannot refer to a non-final variable a inside an inner class defined in a different method]
OOPS460	10	we can refer final variable inside an inner class.

IV. Anonymous class

1. When you define class without specifying the name then it is called as Anonymous class.
2. When you want to declare and instantiate a class at the same time then you can use Anonymous class.
3. Anonymous class definitions can be placed anywhere inside class, blocks, methods or constructors.
4. Anonymous class can be used to write the sub class of an abstract class or an interface.
5. You cannot define constructor for anonymous class because of no name.
6. You cannot define reference variable of anonymous class.
7. Anonymous class will be always subclass of an interface or a class.
8. We cannot create more than one object of anonymous class.
9. We can store object of anonymous class in super type reference variable. You can access only those members with that ref-variable whose signature is available in super class.
10. You can define new members in anonymous class and to access those members you can use dynamic object.

Programs:

OOPS461.java

```
abstract class Person{
public abstract void sleeping();
}
class Student extends Person{
public void sleeping() {
System.out.println("Stud ->sleeping()");
}
}
class OOPS461{
public static void main(String arg[]){
Person p=new Student() ;
p. sleeping();
}
}
```

OOPS462.java

```
abstract class Person{
public abstract void sleeping();
}
class OOPS462{
public static void main(String arg[]){
Person p=new Person() ;
}
}
```

OOPS463.java

```
abstract class Person{
public abstract void sleeping();
}
class OOPS463{
public static void main(String arg[]){
Person p=new Person() {} ;
}
}
```

OOPS464.java

```
abstract class Person{
public abstract void sleeping();
}
class OOPS464{
public static void main(String arg[]){
Person p=new Person() {
public void sleeping ( ){
System.out.println("Student -> sleeping()");
}
};
p. sleeping();
}
}
```

OOPS465.java

```

abstract class Person{
public abstract void sleeping();
}
class Hello{
    Person p=new Person(){
public void sleeping(){
        System.out.println("Stud ->sleeping()");
    }
};
void m1(){
    p. sleeping();
}
class OOPS465{
public static void main(String arg[]){
    Hello h=new Hello();
    h.m1();
    h.p. sleeping();
}
}

```

OOPS466.java

```

abstract class Person{
public abstract void sleeping();
}
class OOPS466{
public static void main(String arg[]){
    Person p=new Person(){
void reading(){
        System.out.println("Stud->reading()");
    }
public void sleeping(){
        System.out.println("Stud->sleeping()");
    }
};
    p. sleeping();
    p. reading();
}
}

```

OOPS467.java

```

abstract class Person{
public abstract void sleeping();
}
class OOPS467{
public static void main(String arg[]){
    Person p=new Person(){
void reading(){
        System.out.println("Stud->reading()");
    }
public void sleeping(){
        System.out.println("Stud->sleeping()");
        reading();
    }
};
    p. sleeping();
}
}

```

OOPS468.java

```

class OOPS426{
public static void main(String arg[]){
abstract class Person{
public abstract void sleeping();
}
class OOPS468{
public static void main(String arg[]){
new Person(){
void reading(){
        System.out.println("Stud->reading()");
    }
public void sleeping(){
        System.out.println("Stud->sleeping()");
    }
}.reading();
}
}
}

```

OOPS469.java

```

import java.io.*;
abstract class Hello{}
class Hai extends Hello{}
interface Ball{}
interface Shape{}
class OOPS470{
public static void main(String arg[]){
    //Hello h1=new Hello();
    //Ball b1=new Ball();
    //Cloneable c=new Cloneable();
    //Serializable s= new Serializable();
    //Shape ss=new Shape();
}
}

```

OOPS470.java

```

import java.io.*;
abstract class Hello{}
class Hai extends Hello{}
interface Ball{}
interface Shape{}
class OOPS470{
public static void main(String arg[]){
    Hello h1=new Hai();
    Hello h2=new Hai();
    Hello h3=new Hello();
    Ball b1=new Ball();
    Cloneable c=new Cloneable();
    Serializable s= new Serializable();
    Shape ss=new Shape();
}
}

```

OOPS471.java

```

class Hello{}
class OOPS471{
public static void main(String arg[]){
Hello h1=new Hello();
Hello h2=new Hello();
System.out.println(h1);
System.out.println(h2);
System.out.println(h1==h2);
}
}

```

OOPS472.java

```

class Hello{}
class OOPS472{
public static void main(String arg[]){
Hello h1=new Hello();
Hello h2=new Hello();
System.out.println(h1);
System.out.println(h2);
System.out.println(h1==h2);
System.out.println(new Hello()==new Hello());
}
}

```

OOPS473.java

```

class OOPS425{
public static void main(String
arg[]){

class A{
int x=10;
}
class B{
char x='k';
}
class Outer extends A{
String x="IND";
class Inner extends B{
boolean x=true;
void show(){

```

```

float x=10.0F;
System.out.println(x);
System.out.println(this.x);
System.out.println(Inner.this.x);
System.out.println(Inner.super.x);
System.out.println(Outer.this.x);
System.out.println(Outer.super.x);
}
}
}
class OOPS473{
public static void main(String arg[]){
new Outer().new Inner().show();
}
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS461	Stud ->sleeping()	Refer page 172 point 1 and 2.
OOPS462	Compilation Error	[Error: Cannot instantiate the type Person] We can't create object for abstract class.
OOPS463	Compilation Error	[Error: The type new Person(){} must implement the inherited abstract method]
OOPS464	Stud -> sleeping()	Here, We are implement the inherited abstract method]
OOPS465	Stud ->sleeping() Stud ->sleeping()	Here, We are implement the inherited abstract method and also creating object for Hello class]
OOPS466	Compilation Error	[Error: The method reading() is undefined for the type Person]
OOPS467	Stud->sleeping() Stud->reading()	Refer page 172 point 3 to 8
OOPS468	Stud->reading()	Refer page 172 point 3 to 8

Prog No.	Output	Error and Details
OOPS469	[Blank]	We can't create object for abstract class and Anonymous class is not cloneable, Serializable.
OOPS470	[Blank]	Refer page 172 point 3 to 8.
OOPS471	OOPS471\$1@1690726 OOPS471\$2@5483cd false	Here, we are creating object for hai class and checking that, is there both object is indicating same address or different address.
OOPS472	Compilation Error	[Error: Incompatible operand types new Hello(){} and new Hello(){}]
OOPS473	10.0 true true k IN 10	Refer page 172 point 9 and 10

Disadvantages of Inner class

- ❖ From a maintenance point of view, inexperienced Java developers may find the inner class difficult to understand.
- ❖ The use of inner classes will also increase the total number of classes in your code

Write your note here.....

~~~~::: GOOD LUCK :::~~~~